Tarea7

Cristina Moreno

1) Dados los puntos (0,1), (1,0), (2,3), determine el spline cúbico

```
def spline_cubico_natural(x, y):
    n = len(x) - 1

    # Inicializar
    h = [x[i+1] - x[i] for i in range(n)]
    b = [(y[i+1] - y[i]) / h[i] for i in range(n)]

    alpha = [0]*(n)
    for i in range(1, n):
        alpha[i] = (3/h[i])*(b[i] - b[i-1]) - (3/h[i-1])*(b[i] - b[i-1])

    l = [0]*(n+1)
    mu = [0]*(n+1)
    z = [0]*(n+1)

    c = [0]*(n+1)
    b = [0]*(n)
    d = [0]*(n)

    for i in range(1, n):
        l[i] = 2*(x[i+1] - x[i-1]) - h[i-1]*mu[i-1]
        mu[i] = h[i]/l[i]
        z[i] = (alpha[i] - h[i-1]*z[i-1])/l[i]

    l[n] = 1
    z[n] = 0
    c[n] = 0

    for j in range(n-1, -1, -1):
        c[j] = z[j] - mu[j]*c[j+1]
        b[j] = (y[j+1] - y[j])/h[j] - h[j]*(c[j+1] + 2*c[j])/3
        d[j] = (c[j+1] - c[j])/(3*h[j])

    return a, b, c, d

x = [0, 1, 2]
y = [1, 0, 3]
a, b, c, d = spline_cubico_natural(x, y)

for j in range(len(x)-1):
    print("Intervalo [%d, %d]:" % (x[j], x[j+1]))
    print("S(x) = %.3f + %.3f*(x - %d) + %.3f*(x - %d)^2 + %.3f*(x - %d)^3" % (a[j], b[j], x[j], c[j], x[j], d[j], x[j]))
```

```
Intervalo [0, 1]:
S(x) = 1.000 + 1.000*(x - 0) + 0.000*(x - 0)^2 + -0.500*(x - 0)^3
Intervalo [1, 2]:
S(x) = 0.000 + 2.000*(x - 1) + -0.500*(x - 1)^2 + 0.500*(x - 1)^3
```

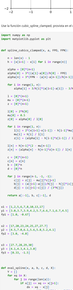
```
import numpy as np
import matplotlib.pyplot as plt

x0 = np.linspace(0, 1, 200)
y0 = np.linspace(1, 0, 200)

def spline1:
    return 1 + 1.5*x0 - 0.5*x0**2

def spline2:
    return 1 - 0.5*(x0-1) - 0.5*(x0-1)**2 + 1.5*(x0-1)**3

plt.figure(figsize=(8, 6))

plt.plot(x0, spline1, label='S(x)', color='#FF0000')
plt.plot(x0, spline2, label='S(x)', color='#FF0000')

x = [0, 1, 2]
y = [1, 0, 3]
plt.scatter(x, y, s=50, color='#000000', zorder=5)

plt.axhline(y=0, color='black', linewidth=0.5)
plt.axvline(x=0, color='black', linewidth=0.5)
plt.xlabel('x', fontsize=12)
plt.ylabel('y', fontsize=12)
plt.legend(fontsize=10)
plt.grid(True)
plt.show()
```

Dados los puntos (-1,5), (1,2), determine el spline cúbico sabiendo que $f'(-1)=1.5$ y $f'(1)=-2$

```
def spline_cubico_sujetado(x, y, FP0, FPN):

    n = len(x) - 1

    h = [x[i+1] - x[i] for i in range(n)]
    b = [(y[i+1] - y[i]) / h[i] for i in range(n)]

    alpha = [0]*(n+1)
    alpha[0] = FP0 - b[0]/h[0]
    alpha[n] = FPN - (y[n] - y[n-1])/h[n-1]

    for i in range(1, n):
        alpha[i] = (3/h[i])*(b[i]-b[i-1]) - 3/h[i-1]*(b[i]-b[i-1])

    l = [0]*(n+1)
    mu = [0]*(n+1)
    z = [0]*(n+1)

    c = [0]*(n+1)
    b = [0]*(n)
    d = [0]*(n)

    for i in range(1, n):
        l[i] = 2*(x[i+1] - x[i-1]) - h[i-1]*mu[i-1]
        mu[i] = h[i]/l[i]
        z[i] = (alpha[i] - h[i-1]*z[i-1])/l[i]

    l[n] = h[n-1]*(2 - mu[n-1])
    z[n] = (alpha[n] - h[n-1]*z[n-1])/l[n]

    c[n] = z[n]
    b[n] = d[n] = 0

    for j in range(n-1, -1, -1):
        c[j] = z[j] - mu[j]*c[j+1]
        b[j] = (y[j+1] - y[j])/h[j] - h[j]*(c[j+1] + 2*c[j])/3
        d[j] = (c[j+1] - c[j])/(3*h[j])

    return a[:-1], b[:-1], c[:-1], d

x = [-1, 1]
y = [5, 2]
FP0 = 1.5
FPN = -2

a, b, c, d = spline_cubico_sujetado(x, y, FP0, FPN)

for j in range(len(x)-1):
    print("Intervalo [%d, %d]:" % (x[j], x[j+1]))
    print("S(x) = %.3f + %.3f*(x - %d) + %.3f*(x - %d)^2 + %.3f*(x - %d)^3" % (a[j], b[j], x[j], c[j], x[j], d[j], x[j]))
```

```
Intervalo [-1, 1]:
S(x) = 5.000 + 1.500*(x - -1) + -0.750*(x - -1)^2 + 0.250*(x - -1)^3
```

```
import numpy as np
import matplotlib.pyplot as plt

x0 = np.linspace(-1, 1, 200)
y0 = np.linspace(5, 2, 200)

def spline1:
    return 5 + 1.5*(x0 - -1) - 0.750*(x0 - -1)^2 + 0.250*(x0 - -1)^3

plt.figure(figsize=(8, 6))

plt.plot(x0, spline1, label='S(x)', color='#FF0000')
plt.plot(x0, spline1, label='S(x)', color='#FF0000')

x = [-1, 1]
y = [5, 2]
plt.scatter(x, y, s=50, color='#000000', zorder=5)

plt.axhline(y=0, color='black', linewidth=0.5)
plt.axvline(x=0, color='black', linewidth=0.5)
plt.xlabel('x', fontsize=12)
plt.ylabel('y', fontsize=12)
plt.legend(fontsize=10)
plt.grid(True)
plt.show()
```



Dados los puntos (-1,5), (1,2), determine el spline cúbico sujetado que $f'(-1)=1.5$ y $f'(1)=-2$

```
def spline_cubico_natural(x, y):
    n = len(x) - 1

    # Inicializar
    h = [x[i+1] - x[i] for i in range(n)]
    b = [(y[i+1] - y[i]) / h[i] for i in range(n)]

    alpha = [0]*(n)
    for i in range(1, n):
        alpha[i] = (3/h[i])*(b[i] - b[i-1]) - (3/h[i-1])*(b[i] - b[i-1])

    l = [0]*(n+1)
    mu = [0]*(n+1)
    z = [0]*(n+1)

    c = [0]*(n+1)
    b = [0]*(n)
    d = [0]*(n)

    for i in range(1, n):
        l[i] = 2*(x[i+1] - x[i-1]) - h[i-1]*mu[i-1]
        mu[i] = h[i]/l[i]
        z[i] = (alpha[i] - h[i-1]*z[i-1])/l[i]

    l[n] = 1
    z[n] = 0
    c[n] = 0

    for j in range(n-1, -1, -1):
        c[j] = z[j] - mu[j]*c[j+1]
        b[j] = (y[j+1] - y[j])/h[j] - h[j]*(c[j+1] + 2*c[j])/3
        d[j] = (c[j+1] - c[j])/(3*h[j])

    return a, b, c, d

x = [-1, 1, 2]
y = [5, 2, 4]

a, b, c, d = spline_cubico_natural(x, y)

for j in range(len(x)-1):
    print("Intervalo [%d, %d]:" % (x[j], x[j+1]))
    print("S(x) = %.3f + %.3f*(x - %d) + %.3f*(x - %d)^2 + %.3f*(x - %d)^3" % (a[j], b[j], x[j], c[j], x[j], d[j], x[j]))
```

```
Intervalo [-1, 1]:
S(x) = 5.000 + 0.000*(x - -1) + 0.000*(x - -1)^2 + 0.000*(x - -1)^3
Intervalo [1, 2]:
S(x) = 2.000 + 2.000*(x - 1) + 3.000*(x - 1)^2 + -3.000*(x - 1)^3
```

```
import numpy as np
import matplotlib.pyplot as plt

x0 = np.linspace(-1, 1, 200)
y0 = np.linspace(5, 2, 200)

def spline1:
    return 5 + 0.000*(x0 - -1) + 0.000*(x0 - -1)^2 + 0.000*(x0 - -1)^3

plt.figure(figsize=(8, 6))

plt.plot(x0, spline1, label='S(x)', color='#FF0000')
plt.plot(x0, spline1, label='S(x)', color='#FF0000')

x = [-1, 1, 2]
y = [5, 2, 4]
plt.scatter(x, y, s=50, color='#000000', zorder=5)

plt.axhline(y=0, color='black', linewidth=0.5)
plt.axvline(x=0, color='black', linewidth=0.5)
plt.xlabel('x', fontsize=12)
plt.ylabel('y', fontsize=12)
plt.legend(fontsize=10)
plt.grid(True)
plt.show()
```

Usando la función anterior, encuentre el spline cúbico para xs = [-1, 1, 2] y ys = [-1, 1, 3]

```
def spline_cubico_natural(x, y):
    n = len(x) - 1

    # Inicializar
    h = [x[i+1] - x[i] for i in range(n)]
    b = [(y[i+1] - y[i]) / h[i] for i in range(n)]

    alpha = [0]*(n)
    for i in range(1, n):
        alpha[i] = (3/h[i])*(b[i] - b[i-1]) - (3/h[i-1])*(b[i] - b[i-1])

    l = [0]*(n+1)
    mu = [0]*(n+1)
    z = [0]*(n+1)

    c = [0]*(n+1)
    b = [0]*(n)
    d = [0]*(n)

    for i in range(1, n):
        l[i] = 2*(x[i+1] - x[i-1]) - h[i-1]*mu[i-1]
        mu[i] = h[i]/l[i]
        z[i] = (alpha[i] - h[i-1]*z[i-1])/l[i]

    l[n] = 1
    z[n] = 0
    c[n] = 0

    for j in range(n-1, -1, -1):
        c[j] = z[j] - mu[j]*c[j+1]
        b[j] = (y[j+1] - y[j])/h[j] - h[j]*(c[j+1] + 2*c[j])/3
        d[j] = (c[j+1] - c[j])/(3*h[j])

    return a, b, c, d

x = [-1, 1, 2]
y = [-1, 1, 3]

a, b, c, d = spline_cubico_natural(x, y)

for j in range(len(x)-1):
    print("Intervalo [%d, %d]:" % (x[j], x[j+1]))
    print("S(x) = %.3f + %.3f*(x - %d) + %.3f*(x - %d)^2 + %.3f*(x - %d)^3" % (a[j], b[j], x[j], c[j], x[j], d[j], x[j]))
```

```
Intervalo [-1, 1]:
S(x) = -1.000 + 0.000*(x - -1) + 0.000*(x - -1)^2 + 0.000*(x - -1)^3
Intervalo [1, 2]:
S(x) = 1.000 + 2.000*(x - 1) + 3.000*(x - 1)^2 + -3.000*(x - 1)^3
```

```
import numpy as np
import matplotlib.pyplot as plt

x0 = np.linspace(-1, 1, 200)
y0 = np.linspace(-1, 3, 200)

def spline1:
    return -1 + 0.000*(x0 - -1) + 0.000*(x0 - -1)^2 + 0.000*(x0 - -1)^3

plt.figure(figsize=(8, 6))

plt.plot(x0, spline1, label='S(x)', color='#FF0000')
plt.plot(x0, spline1, label='S(x)', color='#FF0000')

x = [-1, 1, 2]
y = [-1, 1, 3]
plt.scatter(x, y, s=50, color='#000000', zorder=5)

plt.axhline(y=0, color='black', linewidth=0.5)
plt.axvline(x=0, color='black', linewidth=0.5)
plt.xlabel('x', fontsize=12)
plt.ylabel('y', fontsize=12)
plt.legend(fontsize=10)
plt.grid(True)
plt.show()
```


Usa la función cubic_spline_clamped_procesa en el enlace de GitHub, para graficar los datos de la siguiente tabla

```
import numpy as np
import matplotlib.pyplot as plt

def spline_cubico_clamped(x, y, FP0, FPN):

    n = len(x) - 1
    h = [x[i+1] - x[i] for i in range(n)]

    alpha = [0]*(n+1)
    alpha[0] = FP0*(3/h[0]) - 3/h[0]*(y[1] - y[0])
    alpha[n] = FPN - (3/h[n-1])*(y[n] - y[n-1])

    for i in range(1, n):
        alpha[i] = (3/h[i])*(y[i+1] - y[i]) - 3/h[i-1]*(y[i] - y[i-1])

    l = [0]*(n+1)
    mu = [0]*(n+1)
    z = [0]*(n+1)

    c = [0]*(n+1)
    b = [0]*(n)
    d = [0]*(n)

    l[0] = 2*h[0]
    mu[0] = 0.5
    z[0] = alpha[0]/l[0]

    for i in range(1, n):
        l[i] = 2*(x[i+1] - x[i-1]) - h[i-1]*mu[i-1]
        mu[i] = h[i]/l[i]
        z[i] = (alpha[i] - h[i-1]*z[i-1])/l[i]

    l[n] = h[n-1]*(2 - mu[n-1])
    z[n] = (alpha[n] - h[n-1]*z[n-1])/l[n]

    c[n] = z[n]
    b[n] = d[n] = 0

    for j in range(n-1, -1, -1):
        c[j] = z[j] - mu[j]*c[j+1]
        b[j] = (y[j+1] - y[j])/h[j] - h[j]*(c[j+1] + 2*c[j])/3
        d[j] = (c[j+1] - c[j])/(3*h[j])

    return a[:-1], b[:-1], c[:-1], d

x0 = [1,2,3,4,5,6,7,8,9,10]
y0 = [3,6,8,5,3,0,2,4,3,2,4,5,6,7,8,9,10,12]

S0 = [3,6, 8,4,0]

x1 = [10,11,12,13,14,15,16,17,18]
y1 = [12,14,15,16,17,18,19,20,21]
S1 = [12, 14, 16, 18]

x2 = [18,19,20,21,22,23,24,25,26,27,28,29,30]
y2 = [21,19,18,17,16,15,14,13,12,11,10,9,8]
S2 = [21, 19, 17, 15, 13]

x = x0 + x1 + x2
y = y0 + y1 + y2
S = S0 + S1 + S2

def eval_spline(x, a, b, c, d):
    n = len(x) - 1
    for xp in x:
        for j in range(n):
            if x[j] <= xp <= x[j+1]:
                dx = xp - x[j]
                yp = a[j] + b[j]*dx + c[j]*dx**2 + d[j]*dx**3
                break
    return yp

plt.figure(figsize=(10,4))

for k, p, Sk, color, label in
    [(x0, y0, S0, '#0000ff', 'Curva 1'),
    (x1, y1, S1, '#ff0000', 'Curva 2'),
    (x2, y2, S2, '#008000', 'Curva 3')]:
    a,b,c,d = spline_cubico_clamped(k, p, 0, 0)

    Nk = np.linspace(k[0], k[-1], 200)
    Ys = eval_spline(Nk, k, a, b, c, d)

    plt.plot(Nk, Ys, color=color, label=label)
    plt.scatter(k, p, s=20,color=color)

plt.title('Splines cúbicos con condiciones de contorno (clamped)')
plt.xlabel('x')
plt.ylabel('f(x)')
plt.legend()
plt.grid()
plt.show()
```